Оглавление

Лабораторная работа №3. Тестовое покрытие на графах. 2

Ход работы: 2

Задача 1. 2

Задача 2. 3

Задача 3. 5

Тестовые случаи для кода графа 6

Лабораторная работа №4. Регрессионное тестирование. 8

Ход работы: 8

Задачи 1-3. 8

Задача 4. 9

Задача 5. 10

Задача 6. 10

Вывод: 11

Приложение А.1 12

Приложение А.2 13

Приложение Б.1 14

Приложение Б.2 15

Приложение Б.3 16

Лабораторная работа №3. Тестовое покрытие на графах.

Цель лабораторной работы: изучить принципы разработки вариантов тестового покрытия на примере задач на графах.

Задачи:

1. Из первой лабораторной работы взять любой фрагмент программного кода, в котором встречается логическое условие выбора
2. представить фрагмент в виде графа
3. создать по ГОСТ таблицу тестовых случаев.
4. Провести тестирование.

Ход работы:

Задача 1.

Из первой лабораторной работы взять любой фрагмент программного кода, в котором встречается логическое условие выбора

Был выбран фрагмент кода из лабораторной работы №1, состоящий из трёх условий:

```
// проверка ввода пользователя
function userInputsTest(user_input_n,
                        user_input_m) {
    // Проверка, строка ли это
    if (typeof user_input_n == 'string') {

        // если используется символ
        // возведения в степень (10^6)
        if (value.indexOf('^') !== -1) {
            // преобразует в степень числа
            let index = value.indexOf('^');
            let left = value.slice(0, index);
            let right = value.slice(index+1);
            powered_number = (+left)**(+right);
            return powered_number
        } else {
            throw new Error("Неверная строка")
        }
    } else { return user_input_n }

// Если M это тоже строка
    if (typoef user_input_m == 'string') {
        throw new Error("Ожидалось число.")
    } else {
        return user_input_2
    }
}
```

Рисунок 1 – Условная запись программного кода

Код подразумевает собой проверку пользователем ввода двух чисел:

1. Максимальное количество чисел (user_input_M в условии)
2. Максимальный модуль числа (user_input_N)

Валидация инпута M подразумевает лишь проверку на **число**. Если передано не число, выкидывается ошибка. Таким образом, если на инпут приходит значение, которое представляет собой валидный тип данных (number), то он проходит проверку.

Со вторым инпутом ситуация сложнее. Условие задания подразумевали, что модуль числа может быть большим. Например – 10^9. Записывать это число в виде числового типа данных было бы проблематично, ведь оно должно было бы выглядеть как 1000000000.Потому было разработано условие, при котором, если передана строка, то ошибка появляется не сразу.

Первое условие проверяет тип данных. Если это число, оно и возвращается, и эта часть алгоритма завершает свою работу. Если же это строка, то, прежде чем выкинуть ошибку, проверяется, содержится ли в строке сисмвол «^», который подразумевает возведение в степень. Если нет, то наконец выкидывается ошибка.

Таким образом единственные валидные значения поля — это либо настоящее число (100000) либо строка, в которой обязательно есть символ возведения в степень (10^6), тогда эта строка несложными операциями превращается в число необходимой степени.

Задача 2.

Представить фрагмент кода в виде графа

Для выполнения данной задачи была использована библиотека Fabric.js, позволяющая отрисовывать на холсте различные объекты. Также был разработан класс **GraphNode** (Вершина графа), имеющий в себе методы **draw** (для отрисовки вершины на поле), **remove** (для удаления вершины с

холста), **line** (рисующий связь от текущей ноды до указанной), **checkPathTo** (метод для поиска пути из текущей вершины в указанную).

На основе разработанного класса была создана функция ***initGraph()***:

```
// рисование графа
var InitGraph = () =>
{
    canvas.clear();

    const M = 100; // множитель координат
    var randomSize = (size, delta) => size + Math.random()*delta;

    const powerSuccess = new GraphNode(3*M, 4.5*M, "Возведение\nв степень", 17, successColor, successStroke, 56, []).draw();
    const powerError = new GraphNode(1.5*M, 4.5*M, "Неверная\nстрока", 16, errorColor, errorStroke, 48, []).draw();

    const man2success = new GraphNode(0.75*M, 3*M, "Успех", 20, successColor, successStroke, 36, []).draw();
    const man2error = new GraphNode(2.25*M, 3*M, "Это строка.\nИмеется ли\nсимвол ^ ?", 15, additionalColor, additionalStroke, 56, [powerSuccess, powerError]).draw();

    const man1success = new GraphNode(5.25*M, 2.75*M, "Успех", 20, successColor, successStroke, 36, []).draw();
    const man1error = new GraphNode(3.75*M, 2.75*M, "Ошибка", 18, errorColor, errorStroke, 40, []).draw();

    const man1Node = new GraphNode(4.5*M, 1.5*M, "M - число?", 19, null, null, 50, [man1error, man1success]).draw();
    const man2Node = new GraphNode(1.5*M, 1.5*M, "N - число?", 19, null, null, 56, [man2error, man2success]).draw();
}
```

Рисунок 2 – Инициализация графа

Функция очищает холст, после чего в порядке от листьев к корням графа начинает отрисовывать ноды с некоторыми параметрами по заданным координатам. Таким образом был создан следующий граф:

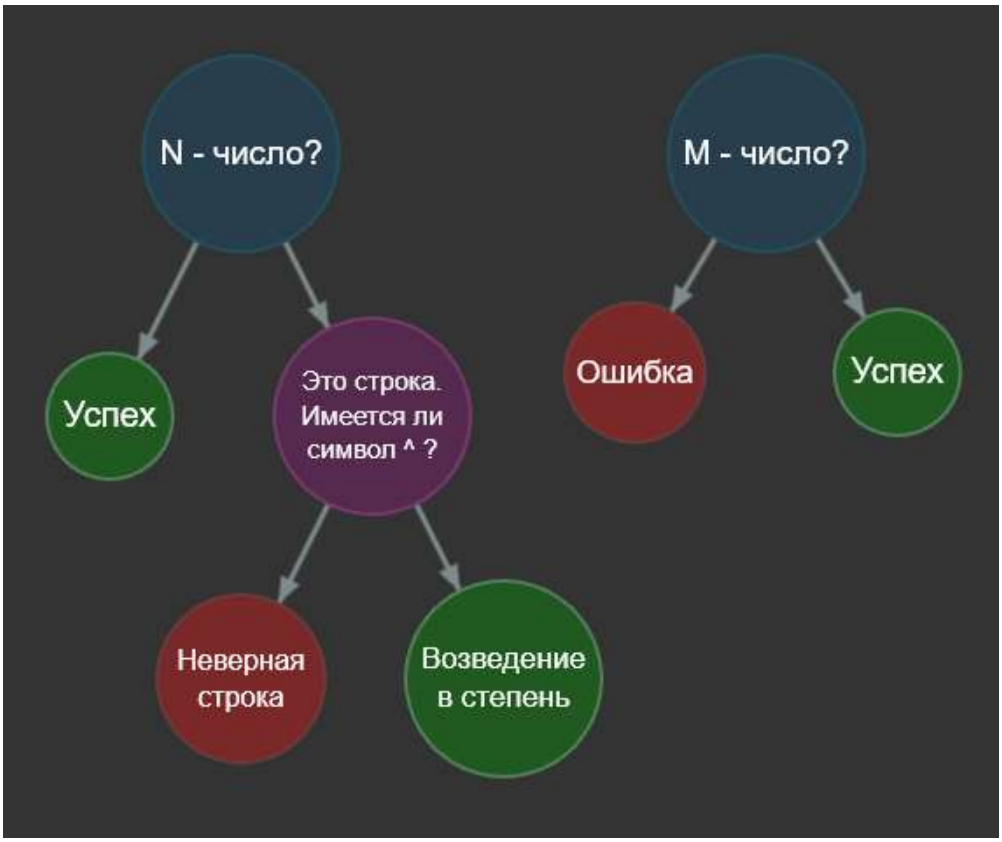


Рисунок 3 – Граф на основе программного кода

Граф и алгоритм, на основе которого он был отрисован можно увидеть по ссылке https://kaixotwentytwo.github.io/Testing-Software/lab3.html

Задача 3.

Объект испытаний: Класс GraphNode, реализующий вершины ориентированного графа.

• Цель испытаний: Проверка корректности создания графа, добавления рёбер и вывода списка смежности

• Требования к программе:

o Граф создается с заданным количеством вершин

o Поддерживается добавление ориентированных рёбер

o Метод printGraph выводит корректный список смежности

• Средства и порядок испытаний:

o Тестирование проводится на HTML-странице в браузере

o Тестовые случаи реализуются как функции с проверкой вывода или состояния списка смежности

• Методы испытаний:

o Функциональное тестирование методом черного ящика с проверкой соответствия результата ожиданиям

Тестовые случаи для кода графа

№ Теста	Название теста	Входные данные	Ожидаемый результат	Цель теста
1	Создание пустого графа	Создать граф с нулём вершин	Пустой список смежности (ничего не выводит)	Проверка корректного создания пустого графа
2	Создание графа с вершинами и без рёбер	Создать граф с 3 вершинами, 0 рёбрами	Для каждой вершины пустой список смежности	Проверка инициализации без рёбер
3	Добавление одного ребра	Добавить ребро 1→2 в граф с 3 вершинами	Вершина 1: 2; Остальные вершины без смежных	Проверка добавления одного ребра
4	Добавление нескольких рёбер	Добавить рёбра согласно примеру кода (1→3, 1→7, 3→1...) для 8 вершин	Корректный вывод списка смежности, совпадающий с графом	Проверка наполнения списка смежности
5	Корректность вывода	Вызов метода draw() после добавления рёбер	Вывод строк с вершинами и соседями как в примере	Проверка корректности отрисовки
6	Обработка некорректных рёбер	Добавить ребро с индексами за пределами	Не происходит сбоев	Проверка устойчивости к ошибкам ввода

Вывод и тд

Лабораторная работа №4. Регрессионное тестирование.

Цель лабораторной работы: освоить навыки регрессионного тестирования.

Задачи:

1. Создать вектор с замерами времени сортируется по возрастанию с помощью стандартной библиотеки: std::sort(latencies.begin(), latencies.end());
2. Вычисляется индекс нужного элемента в массиве. Для P95 это 95% от общего количества запросов: size_t index95 = latencies.size() * 0.95;
3. Значение из массива по этому индексу и есть 95-й перцентиль: double p95 = latencies[index95];
4. Проверка масштабируемости на 50 и 500 потоках.
5. Добавить метрику P99
6. Добавление механизма ступенчатого увеличения нагрузки

Ход работы:

Задачи 1-3.

В соответствии с первыми тремя задачами был написан код (Приложение А), выводящий в консоль следующий результат:

```
calculating latency...
progress: 1000/10000 (10%), avg latency: 80.7042 ms
progress: 2000/10000 (20%), avg latency: 79.873 ms
progress: 3000/10000 (30%), avg latency: 80.1412 ms
progress: 4000/10000 (40%), avg latency: 80.1731 ms
progress: 5000/10000 (50%), avg latency: 80.366 ms
progress: 6000/10000 (60%), avg latency: 80.3586 ms
progress: 7000/10000 (70%), avg latency: 80.2829 ms
progress: 8000/10000 (80%), avg latency: 80.3787 ms
progress: 9000/10000 (90%), avg latency: 80.3685 ms
progress: 10000/10000 (100%), avg latency: 80.3631 ms

result:
threads: 50
latency size: 10000
```

```
tps: 618.517 operation/s
p95: 143.088 ms
p99: 149.071 ms
```

Задача 4.

Проверка результатов при использовании 500 потоков и 10.000 запросов

```
calculating latency...
pthread_create #89 failed: Resource temporarily unavailable
warning: created only 89 of 500 threads
progress: 1000/10000 (10%), avg latency: 77.3133 ms
progress: 2000/10000 (20%), avg latency: 78.8028 ms
progress: 3000/10000 (30%), avg latency: 79.0057 ms
progress: 4000/10000 (40%), avg latency: 79.4575 ms
progress: 5000/10000 (50%), avg latency: 79.7117 ms
progress: 6000/10000 (60%), avg latency: 79.597 ms
progress: 7000/10000 (70%), avg latency: 79.5885 ms
progress: 8000/10000 (80%), avg latency: 79.4913 ms
progress: 9000/10000 (90%), avg latency: 79.614 ms
progress: 10000/10000 (100%), avg latency: 79.9035 ms

result:
threads: 500
latency size: 10000
tps: 1102.38 operation/s
p95: 143.079 ms
p99: 149.071 ms
```

При сравнении с предыдущими результатами, наиболее значительная разница заметна на примере параметра `tps`

```
threads: 50
tps: 618

threads 500
tps: 1102
```

Задача 5.

Показания метрики P99 при 50:

```
threads: 50
p95: 143.088 ms
p99: 149.071 ms
```

Задача 6.

Реализация Ramp-Up (Приложение Б)

Результаты:

```
stepwise load test: 10 steps, up to 10000 requests, 50 threads per step

=== step 1/10: +1000 requests (total 1000) ===
  requests: 1000, time: 1.72762 s, tps: 578.83, avg: 81.4173 ms, p95:
143.07 ms, p99: 150.066 ms

=== step 2/10: +1000 requests (total 2000) ===
  requests: 1000, time: 1.64551 s, tps: 607.714, avg: 78.3871 ms, p95:
142.067 ms, p99: 148.077 ms

=== step 3/10: +1000 requests (total 3000) ===
  requests: 1000, time: 1.68307 s, tps: 594.153, avg: 79.8793 ms, p95:
144.068 ms, p99: 149.07 ms

=== step 4/10: +1000 requests (total 4000) ===
  requests: 1000, time: 1.69254 s, tps: 590.828, avg: 80.0268 ms, p95:
144.081 ms, p99: 149.069 ms

=== step 5/10: +1000 requests (total 5000) ===
  requests: 1000, time: 1.66832 s, tps: 599.404, avg: 79.2839 ms, p95:
144.068 ms, p99: 149.076 ms

=== step 6/10: +1000 requests (total 6000) ===
  requests: 1000, time: 1.67625 s, tps: 596.569, avg: 79.2378 ms, p95:
142.085 ms, p99: 149.069 ms

=== step 7/10: +1000 requests (total 7000) ===
```

```
        requests: 1000, time: 1.69352 s, tps: 590.486, avg: 80.1915 ms, p95:
142.066 ms, p99: 149.019 ms

      === step 8/10: +1000 requests (total 8000) ===
        requests: 1000, time: 1.69128 s, tps: 591.268, avg: 79.7878 ms, p95:
144.071 ms, p99: 150.067 ms

      === step 9/10: +1000 requests (total 9000) ===
        requests: 1000, time: 1.67359 s, tps: 597.517, avg: 79.5522 ms, p95:
142.072 ms, p99: 148.09 ms

      === step 10/10: +1000 requests (total 10000) ===
        requests: 1000, time: 1.62968 s, tps: 613.616, avg: 77.0131 ms, p95:
143.068 ms, p99: 149.069 ms

      === overall ===
      steps: 10
      threads per step: 50
      total requests: 10000
      total time: 16.7842 s
      tps: 595.797 operation/s
      avg: 79.4777 ms
      p95: 143.076 ms
      p99: 149.072 ms
```

Вывод:

В ходе выполнения работы было разработано консольное приложение, имитирующее нагрузку на систему и отображающее оценку производительности системы. Тестирование было проведено при различном количестве задействованных потоков. Были отображены такие метрики как: latency, tps, p95, p99, также был реализован оптимизационный механизм Ramp-Up.

Приложение А.1

Регрессионное тестирование. Листинг кода для задач 1-5

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <chrono>
#include <thread>
#include <random>
#include <atomic>
#include <mutex>
#include <functional>
#include <cstring>
#include <pthread.h>
const int num_threads = 500;
const int total_requests = 10000;
const size_t thread_stack_size = 256 * 1024; // 256 КБ
void do_work() {
    static thread_local std::random_device rd;
    static thread_local std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(10, 150);
    std::this_thread::sleep_for(std::chrono::milliseconds(dis(gen)));
}
struct ThreadArg {
    std::function<void()> fn;
};
static void* thread_entry(void* p) {
    auto* arg = static_cast<ThreadArg*>(p);
    try {
        arg->fn();
    } catch (...) {
    }
    delete arg;
    return nullptr;
}
int main() {
    std::vector<double> latencies;
    latencies.reserve(total_requests);
    std::mutex latencies_mutex;
    const int step = std::max(1, total_requests / 10);
    std::cout << "calculating latency...\n";
    auto total_start = std::chrono::high_resolution_clock::now();
    std::atomic<int> next_request{0};
    std::atomic<int> completed{0};
    pthread_attr_t attr;
    if (pthread_attr_init(&attr) != 0) {
        std::cerr << "pthread_attr_init failed\n";
        return 1;
    }
    if (pthread_attr_setstacksize(&attr, thread_stack_size) != 0) {
        std::cerr << "pthread_attr_setstacksize failed\n";
        pthread_attr_destroy(&attr);
        return 1;
    }
    std::vector<pthread_t> threads(num_threads);
    int created = 0;
    auto worker_body = [&]() {
        while (true) {
            int i = next_request.fetch_add(1);
            if (i >= total_requests) break;
            auto start = std::chrono::high_resolution_clock::now();
            do_work();
            auto end = std::chrono::high_resolution_clock::now();
            std::chrono::duration<double, std::milli> duration = end - start;
            {
                std::lock_guard<std::mutex> lock(latencies_mutex);
                latencies.push_back(duration.count());
                int done = completed.fetch_add(1) + 1;
```

Приложение А.2

```
                if (done % step == 0 || done == total_requests) {
                    double sum = 0.0;
                    for (double v : latencies) sum += v;
                    double avg = sum / latencies.size();
                    double percent = 100.0 * done / total_requests;

                    std::cout << "progress: " << done << "/" << total_requests
                              << " (" << percent << "%), avg latency: "
                              << avg << " ms" << std::endl;
                }
            }
        }
    };

    for (int t = 0; t < num_threads; ++t) {
        auto* arg = new ThreadArg{ worker_body };
        int rc = pthread_create(&threads[t], &attr, thread_entry, arg);
        if (rc != 0) {
            std::cerr << "pthread_create #" << t
                      << " failed: " << std::strerror(rc) << "\n";
            delete arg;
            break;
        }
        ++created;
    }

    if (created < num_threads) {
        std::cerr << "warning: created only " << created
                  << " of " << num_threads << " threads\n";
    }

    // дожидаемся только реально созданных потоков
    for (int t = 0; t < created; ++t) {
        pthread_join(threads[t], nullptr);
    }

    pthread_attr_destroy(&attr);

    // если из-за сбоя создания потоков остались невыполненные запросы —
    // добираем их в главном потоке
    if (completed.load() < total_requests) {
        std::cerr << "warning: " << (total_requests - completed.load())
                  << " requests not executed by pool, running in main thread\n";
        worker_body();
    }

    std::sort(latencies.begin(), latencies.end());
    size_t index95 = static_cast<size_t>(std::round((latencies.size() - 1) * 0.95));
    size_t index99 = static_cast<size_t>(std::round((latencies.size() - 1) * 0.99));
    double p95 = latencies[index95];
    double p99 = latencies[index99];

    // расчёт tps
    auto total_end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> total_duration_seconds = total_end - total_start;
    double total_seconds = total_duration_seconds.count();
    double tps = total_requests / total_seconds;

    std::cout << "\nresult:" << std::endl;
    std::cout << "threads: " << num_threads << std::endl;
    std::cout << "latency size: " << latencies.size() << std::endl;
    std::cout << "tps: " << tps << " operation/s" << std::endl;
    std::cout << "p95: " << p95 << " ms" << std::endl;
    std::cout << "p99: " << p99 << " ms" << std::endl;

    return 0;
}
```

Приложение Б.1

Регрессионное тестирование. Листинг кода для задачи 6

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <chrono>
#include <thread>
#include <random>
#include <atomic>
#include <mutex>
#include <functional>
#include <cstring>
#include <pthread.h>
const int num_threads = 50;
const int total_requests = 10000;
const int num_steps = 10;
const size_t thread_stack_size = 256 * 1024; // 256 КБ
void do_work() {
    static thread_local std::random_device rd;
    static thread_local std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(10, 150);
    std::this_thread::sleep_for(std::chrono::milliseconds(dis(gen)));}
struct ThreadArg {    std::function<void()> fn;};
static void* thread_entry(void* p) {
    auto* arg = static_cast<ThreadArg*>(p);
    try {arg->fn();catch (...) {}
    delete arg;
    return nullptr;}
struct Stats {
    double avg = 0.0;
    double p95 = 0.0;
    double p99 = 0.0;
    double tps = 0.0;
    size_t count = 0;};
Stats compute_stats(std::vector<double> latencies, double seconds) {
    Stats s;
    s.count = latencies.size();
    if (latencies.empty()) return s;
    double sum = 0.0;
    for (double v : latencies) sum += v;
    s.avg = sum / latencies.size();
    std::sort(latencies.begin(), latencies.end());
    size_t i95 = static_cast<size_t>(std::round((latencies.size() - 1) * 0.95));
    size_t i99 = static_cast<size_t>(std::round((latencies.size() - 1) * 0.99));
    s.p95 = latencies[i95];
    s.p99 = latencies[i99];
    if (seconds > 0.0) {s.tps = latencies.size() / seconds; return s;}
```

Приложение Б.2

```
        int main() {
            std::cout << "stepwise load test: " << num_steps << " steps, "
                      << "up to " << total_requests << " requests, "
                      << num_threads << " threads per step\n\n";
            auto overall_start = std::chrono::high_resolution_clock::now();
            std::vector<double> all_latencies;
            all_latencies.reserve(total_requests);
            pthread_attr_t attr;
            if (pthread_attr_init(&attr) != 0) {
                std::cerr << "pthread_attr_init failed\n";
                return 1;    }
            if (pthread_attr_setstacksize(&attr, thread_stack_size) != 0) {
                std::cerr << "pthread_attr_setstacksize failed\n";
                pthread_attr_destroy(&attr);
                return 1;    }
            for (int step_idx = 1; step_idx <= num_steps; ++step_idx) {
                const int target_total = total_requests * step_idx / num_steps;
                const int step_requests = target_total - (total_requests * (step_idx - 1) /
num_steps);
                std::cout << "=== step " << step_idx << "/" << num_steps
                          << ": +" << step_requests << " requests (total "
                          << target_total << ") ===\n";
                std::vector<double> step_latencies;
                step_latencies.reserve(step_requests);
                std::mutex latencies_mutex;
                std::atomic<int> next_request{0};
                std::atomic<int> completed{0};
                auto step_start = std::chrono::high_resolution_clock::now();
                auto worker_body = [&]() {
                    while (true) {
                        int i = next_request.fetch_add(1);
                        if (i >= step_requests) break;
                        auto start = std::chrono::high_resolution_clock::now();
                        do_work();
                        auto end = std::chrono::high_resolution_clock::now();
                        std::chrono::duration<double, std::milli> duration = end - start;
                        {std::lock_guard<std::mutex> lock(latencies_mutex);
                            step_latencies.push_back(duration.count());
                            completed.fetch_add(1);} }};
                const int step_threads = std::min(num_threads, step_requests);
                std::vector<pthread_t> threads(step_threads);
                int created = 0;
```

Приложение Б.3

```
        for (int t = 0; t < step_threads; ++t) {
            auto* arg = new ThreadArg{ worker_body };
            int rc = pthread_create(&threads[t], &attr, thread_entry, arg);
            if (rc != 0) {
                std::cerr << "  pthread_create #" << t
                          << " failed: " << std::strerror(rc) << "\n";
                delete arg;
                break;}
            ++created;}
        if (created < step_threads) {
            std::cerr << "  warning: created only " << created
                      << " of " << step_threads << " threads\n";}
        for (int t = 0; t < created; ++t) {pthread_join(threads[t], nullptr);}
        if (completed.load() < step_requests) {
            std::cerr << "  warning: " << (step_requests - completed.load())
                      << " requests not executed by pool, running in main thread\n";
            worker_body();}
        auto step_end = std::chrono::high_resolution_clock::now();
        double step_seconds =
            std::chrono::duration<double>(step_end - step_start).count();
        all_latencies.insert(all_latencies.end(),
                             step_latencies.begin(), step_latencies.end());
        Stats s = compute_stats(std::move(step_latencies), step_seconds);
        std::cout << "  requests: " << s.count
                  << ", time: " << step_seconds << " s"
                  << ", tps: " << s.tps
                  << ", avg: " << s.avg << " ms"
                  << ", p95: " << s.p95 << " ms"
                  << ", p99: " << s.p99 << " ms\n\n";}
    pthread_attr_destroy(&attr);
    auto overall_end = std::chrono::high_resolution_clock::now();
    double overall_seconds =
        std::chrono::duration<double>(overall_end - overall_start).count();
    Stats overall = compute_stats(std::move(all_latencies), overall_seconds);
    std::cout << "=== overall ===\n";
    std::cout << "steps: " << num_steps << "\n";
    std::cout << "threads per step: " << num_threads << "\n";
    std::cout << "total requests: " << overall.count << "\n";
    std::cout << "total time: " << overall_seconds << " s\n";
    std::cout << "tps: " << overall.tps << " operation/s\n";
    std::cout << "avg: " << overall.avg << " ms\n";
    std::cout << "p95: " << overall.p95 << " ms\n";
    std::cout << "p99: " << overall.p99 << " ms\n";
    return 0;
}
```